

Pipeline de données Big Data

Ingestion API → Data Lake (HDFS) → Spark/YARN → Data Warehouse
(PostgreSQL)

Orchestration Airflow • Docker • CI/CD GitHub Actions • AWS EC2

Rémi Descamps

Atelier : Déploiement de pipeline (Hadoop)

Dépôt du projet : github.com/rere1208/hadoop-pipeline

10 septembre 2026

1. Contexte et objectifs

Ce projet répond à l'atelier noté « Déploiement de pipeline (Hadoop) ». L'objectif est de concevoir, déployer et industrialiser un pipeline de données complet, de l'ingestion d'une source de données jusqu'au déploiement cloud, en mettant en pratique les briques classiques d'un écosystème Big Data : stockage distribué (Data Lake HDFS), traitement distribué (Spark sur YARN), restitution analytique (Data Warehouse SQL), orchestration (Airflow), conteneurisation (Docker) et intégration/déploiement continu (CI/CD).

Le sujet de données était libre, avec une contrainte explicite du formateur : les données devaient être « **en continu** » (un flux vivant, pas un jeu de données statique téléchargé une fois). Le choix s'est porté sur l'API publique **Open-Meteo** (météo et qualité de l'air), gratuite, sans clé, et dont les valeurs évoluent en temps réel — ce qui permet de démontrer un vrai pipeline d'ingestion récurrente (DAG Airflow planifié @daily) plutôt qu'un simple traitement ponctuel.

2. Démarche

- **Cadrage** : traduction du cahier des charges (ingestion API, Data Lake, transformation Spark, Data Warehouse SQL propre, orchestration Airflow, Docker, CI/CD, déploiement cloud) en une architecture concrète, avant d'écrire le moindre code.
- **Choix techniques pragmatiques** : réutilisation d'images Docker Hadoop/YARN éprouvées (bde2020) plutôt que de tout construire from scratch ; Spark en `--deploy-mode client` (pas de cluster Spark dédié) pour que les jobs apparaissent directement comme applications YARN, conformément à l'exigence de preuve « jobs YARN réussis ».
- **Développement itératif** : scaffolding du dépôt et des scripts, validation isolée du cluster Hadoop, puis ajout progressif de l'ingestion, de la transformation Spark, du chargement SQL et enfin de l'orchestration Airflow.
- **Validation systématique** : exécution réelle du pipeline en local (pas seulement une relecture de code), avec vérification à chaque étape via les interfaces web (NameNode, YARN, Airflow) et des requêtes SQL directes.
- **Mise en place du CI/CD** dès que le code a été stabilisé, pour garantir la non-régression (lint, tests unitaires, build de l'image Docker) à chaque évolution.

3. Description du cas d'usage

Pipeline de données complet qui collecte, en continu, la météo et la qualité de l'air de plusieurs villes françaises via l'API publique **Open-Meteo** (données en temps réel, mises à jour en permanence — pas un jeu de données statique), stocke les données brutes dans un **Data Lake HDFS**, les nettoie et les transforme avec **Apache Spark** (soumis sur **YARN**), puis les charge dans un **Data Warehouse PostgreSQL** exposé via des vues SQL propres et analysables. L'ensemble est orchestré par **Apache Airflow** (planification quotidienne @daily, historique accumulé jour après jour), conteneurisé avec **Docker Compose**, testé et validé automatiquement via **GitHub Actions (CI/CD)**, et déployable sur **AWS EC2**.

Dépôt du projet (code source complet, README détaillé) :

<https://github.com/rere1208/hadoop-pipeline>

4. Architecture technique

Flux de données de bout en bout :

```
Open-Meteo API (meteo + qualite de l'air)
  | ingestion/extract_openmeteo.py
  v
JSON brut (local, horodate)
  | upload WebHDFS (etl/hdfs_utils.py)
  v
HDFS /data-lake/raw/weather/dt=YYYY-MM-DD/          <- Data Lake
  | spark-submit --master yarn --deploy-mode client
  | spark_jobs/transform_weather.py
  v
HDFS /data-lake/processed/weather/dt=YYYY-MM-DD/ (Parquet)
  | etl/load_to_postgres.py
  v
PostgreSQL (postgres-dwh)
  staging_weather -> vues clean_weather / daily_weather_summary <- Data Warehouse
  ^
  +-- orchestre de bout en bout par Airflow (DAG @daily)
```

Composants Docker (réseau hadoop-net)

Service	Image / build	Rôle	Port hôte
namenode	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java11	HDFS NameNode	9870, 9000
datanode	bde2020/hadoop-datanode:...	HDFS DataNode	—
resourcemanager	bde2020/hadoop-resourcemanager:...	YARN ResourceManager	8088
nodemanager	bde2020/hadoop-nodemanager:...	YARN NodeManager	8042
historyserver	bde2020/hadoop-historyserver:...	YARN Timeline/History	8188
postgres-dwh	postgres:16	Data Warehouse	5433
postgres-airflow	postgres:16	Métadonnées Airflow	—
airflow-webserver	build local (airflow/Dockerfile)	UI Airflow	8080
airflow-scheduler	build local (airflow/Dockerfile)	Scheduler + driver Spark	—

L'image `airflow-spark (custom)` embarque Java 17, Spark 3.5.9 (build Hadoop 3) et les clients Python HDFS/PostgreSQL. Le job Spark est soumis en `--deploy-mode client` : le driver tourne dans ce conteneur, les exécuteurs sont lancés par YARN sur le `nodemanager`. La transformation n'utilisant aucune UDF Python, aucun interpréteur Python n'est requis côté exécuteur — ce qui évite d'installer Spark/Python dans les images Hadoop du cluster.

5. Prérequis

- Docker Desktop (avec WSL2 sous Windows) — Docker Engine ≥ 24, Compose v2

- **16 Go de RAM minimum** allouée à Docker (stack HDFS+YARN+Spark+Airflow+Postgres gourmand)
- Ports libres sur l'hôte : 8080, 8088, 8042, 8188, 9870, 9000, 5433
- Un compte GitHub (dépôt) et, pour le déploiement, un compte AWS

Aucune installation locale de Java/Hadoop/Spark n'est nécessaire : tout tourne dans les conteneurs.

6. Installation et déploiement (local)

```
git clone https://github.com/rere1208/hadoop-pipeline.git
cd hadoop-pipeline
cp .env.example .env
```

```
docker compose up -d --build
```

Puis initialiser l'arborescence du Data Lake :

```
./scripts/init_hdfs_dirs.sh
```

Vérifier que tout tourne : `docker compose ps`. Pour un déploiement sur AWS EC2, voir `deploy/aws/DEPLOY.md` (scripts prêts, non exécutés pour ce rendu afin d'éviter les coûts AWS).

Déploiement effectivement réalisé : au-delà du local, le pipeline a été déployé et exécuté de bout en bout sur un serveur Linux distant (Docker déjà installé), en dehors de la machine de développement — conteneurs tous healthy, DAG Airflow 5/5 en succès, job Spark FINISHED / SUCCEEDED sur YARN. Les ports en conflit avec des services déjà présents sur ce serveur partagé ont été remappés via les variables `${AIRFLOW_WEBSERVER_PORT}` / `${NAMENODE_RPC_PORT}` du `docker-compose.yml`, sans impacter les services déjà en place. Voir la section 10 pour les captures d'écran de ce déploiement distant.

7. Guide d'exécution

Déclencher le pipeline

Via l'UI Airflow (<http://localhost:8080>) : activer puis déclencher le DAG `weather_pipeline_dag`. Ou en ligne de commande :

```
docker exec airflow-webserver airflow dags trigger weather_pipeline_dag
```

Vérifier chaque étape

- **Data Lake (HDFS)** : <http://localhost:9870> → Utilities > Browse the file system
- **Job Spark sur YARN** : <http://localhost:8088> → application avec l'état FINISHED / SUCCEEDED
- **Data Warehouse (PostgreSQL)** : `SELECT * FROM daily_weather_summary;`

8. CI/CD

Pipeline GitHub Actions (`.github/workflows/`) :

- **ci.yml** (automatique, à chaque push/PR sur main) : lint (ruff) → test (pytest) → build (image Docker)
- **deploy.yml** (déclenchement manuel uniquement) : déploiement SSH vers une instance EC2

Un pipeline GitLab CI équivalent (`.gitlab-ci.yml`) est également conservé dans le dépôt.

9. Difficultés rencontrées et solutions apportées

La mise en production réelle du pipeline (et pas seulement sa conception sur le papier) a fait apparaître plusieurs problèmes concrets, résolus au fil de l'implémentation :

Téléchargement Spark bloqué en CI (25+ minutes)

Le job de build GitHub Actions restait bloqué sur le téléchargement de Spark : la version choisie initialement (3.2.1, pour coller à la version exacte de Hadoop du cluster) n'est plus disponible que sur `archive.apache.org`, un serveur d'archives très lent pour les gros fichiers. **Solution** : migration vers Spark 3.5.9, toujours activement distribué sur le miroir rapide `d1cdn.apache.org`, avec Java 17 en cohérence — le build est passé de 25+ minutes (bloqué) à moins de 2 minutes.

Paquet Java absent de l'image de base

L'image Debian de base d'Airflow ne fournit plus de paquet `openjdk` complet pour les anciennes versions de Java (ni JDK ni JRE). **Solution** : téléchargement direct du JRE Eclipse Temurin via l'API Adoptium plutôt que de dépendre des paquets `apt` disponibles, qui varient selon la version de Debian de l'image de base.

Démarrage en échec du webserver/scheduler Airflow

Au premier lancement, `airflow-webserver` et `airflow-scheduler` plantaient immédiatement (« You need to initialize the database »). Cause : une dépendance Docker Compose sans condition n'attend que le *démarrage* du conteneur d'initialisation, pas sa *fin* — les deux services démarraient donc avant que la migration de la base de métadonnées Airflow soit terminée. **Solution** : dépendance condition: `service_completed_successfully` pour forcer l'attente de la fin réelle de l'initialisation.

Incompatibilité pandas / SQLAlchemy lors du chargement en base

Le chargement des données transformées vers PostgreSQL échouait avec `AttributeError: 'Connection' object has no attribute 'cursor'` : une incompatibilité connue entre pandas 2.2 et la version de SQLAlchemy imposée par Airflow (1.4), qui fait planter `DataFrame.to_sql()`. **Solution** : remplacement par un insert direct via `psycopg2.extras.execute_values`, ce qui a aussi permis de retirer une dépendance devenue inutile.

Authentification Git/GitHub bloquée

Le push initial vers GitHub échouait systématiquement (« Password authentication is not supported ») à cause d'un identifiant obsolète mis en cache par le gestionnaire d'identifiants Windows. **Solution** : génération d'un Personal Access Token GitHub et nouvelle authentification via le flux OAuth du Git Credential Manager.

Ces difficultés, bien que non anticipées lors de la conception initiale, illustrent l'écart habituel entre une architecture qui « devrait marcher sur le papier » et sa mise en œuvre réelle — et l'intérêt de valider chaque brique par une exécution effective plutôt que par une simple relecture de code.

Pipeline exécuté et vérifié de bout en bout le 10 septembre 2026 : les 5 tâches du DAG Airflow se sont terminées avec succès, les 3 jobs Spark soumis sur YARN sont **FINISHED / SUCCEEDED**, et les données réelles de 5 villes françaises sont visibles, nettoyées et agrégées, dans le Data Warehouse.

[illegible]

NameNode HDFS — cluster actif, 1 DataNode live

Hadoop |
 Overview |
 Datanodes |
 Datanode Volume Failures |
 Snapshots |
 Startup Progress |
 Utilities >

Browse Directory

Go!

Show 25 entries

	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	drwxrwxrwx	root	supergroup	0 B	Sep 10 10:41	2	0 B	processed	
☐	drwxrwxrwx	root	supergroup	0 B	Sep 10 10:40	2	0 B	raw	

Showing 1 to 2 of 2 entries

[Previous](#)
[Next](#)

Hadoop, 2019

Data Lake HDFS — dossiers raw/ et processed/



Résultat SQL — *SELECT * FROM daily_weather_summary;*

11. Conclusion et bilan

Le pipeline répond à l'ensemble des exigences du cahier des charges : ingestion depuis une API de données continues, stockage brut dans un Data Lake HDFS, transformation via un job Spark soumis sur YARN, chargement dans un Data Warehouse PostgreSQL exposé par des vues SQL propres, orchestration Airflow planifiée quotidiennement, conteneurisation Docker complète, et CI/CD automatisé. Il a été exécuté et vérifié de bout en bout — en local, puis sur un serveur distant réel (section 10) — pas seulement conçu sur le papier.

Ce projet a permis de mettre en pratique l'articulation concrète entre les briques d'un écosystème Big Data (HDFS, YARN, Spark) et les outils d'industrialisation qui les entourent (Docker, Airflow, CI/CD) — et surtout de rencontrer et résoudre des problèmes réels d'intégration (versions incompatibles, ordonnancement de conteneurs, lenteur réseau) qui n'apparaissent qu'à l'exécution effective du système.

Pistes d'amélioration

- **Déploiement AWS EC2** : les scripts et la documentation sont prêts (`deploy/aws/`) mais n'ont pas été exécutés, pour éviter un coût d'infrastructure inutile — le déploiement réel (section 6/10) a été fait sur un serveur Linux distant déjà disponible plutôt que sur une instance AWS payante.
- **Idempotence du chargement** : gérer les ré-exécutions manuelles (upsert plutôt qu'insert simple) pour éviter les conflits de clé primaire en cas de relance du même jour.
- **Observabilité** : ajouter des alertes Airflow (email/Slack) en cas d'échec de tâche pour un usage en production continue.

12. Arborescence du projet

```
hadoop-pipeline/  
  docker-compose.yml      services Hadoop, Spark client, Airflow, Postgres  
  hadoop.env              config du cluster HDFS/YARN (images bde2020)  
  airflow/  
    Dockerfile            image Airflow + Java + Spark + libs  
    conf/                 core-site.xml / hdfs-site.xml / yarn-site.xml  
    dags/weather_pipeline_dag.py  
  ingestion/extract_openmeteo.py  
  spark_jobs/transform_weather.py  
  etl/  hdfs_utils.py, load_to_postgres.py  
  sql/  01_create_schema.sql, 02_clean_views.sql  
  scripts/init_hdfs_dirs.sh  
  tests/  
  deploy/aws/  user-data.sh, DEPLOY.md  
  screenshots/  
  .github/workflows/  ci.yml, deploy.yml  
  .gitlab-ci.yml
```